UE24CS242B
OS: Unit-4 Orange Problem

Name: Abhiraam Adiga **SRN:** PES1UG24CS015

Phase 1

1b

```
adiga@adiga:~/Desktop/OS_Orange$ cd /home/adiga/Desktop/OS_Orange/os-u4-orange-p
roblem
./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1b

```
adiga@adiga:~/Desktop/OS_Orange/os-u4-orange-problem$ cd /home/adiga/Desktop/OS_
Orange/os-u4-orange-problem
find .pes/objects -type f | sort
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2

2a

```
adiga@adiga:~/Desktop/OS_Orange/os-u4-orange-problem$ cd /home/adiga/Desktop/OS_
Orange/os-u4-orange-problem
make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2b

```
adiga@adiga:~/Desktop/OS_Orange/os-u4-orange-problem$ rm -rf .pes
mkdir -p .pes/objects .pes/refs/heads
printf "100644 1111111111111111111111111111111111111111111111111111111111111111
0 5 README.md\n100644 2222222222222222222222222222222222222222222222222222222222
222222 0 7 src/main.c\n" > .pes/index
cat > gen_tree_driver.c <<'EOF'
#include "pes.h"
#include <stdio.h>
int tree_from_index(ObjectID *id_out);
int main(void) {
ObjectID id;
if (tree_from_index(&id) != 0) return 1;
char hex[HASH_HEX_SIZE + 1];
hash_to_hex(&id, hex);
printf("%s\n", hex);
return 0;
}
EOF
gcc -Wall -Wextra -O2 -o gen_tree_driver gen_tree_driver.c tree.c object.c -lcry
pto
./gen_tree_driver
f8c74b2cecc833c1d8d3281c252f1b877321e9a96dfb892afb8ed6a4730ef9e8
```

Phase 3

3a

```
adiga@adiga:~/Desktop/OS_Orange/os-u4-orange-problem$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  pes.c
  untracked:  commit.h
  untracked:  commit.c
  untracked:  gen_tree_driver.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  tree.c
  untracked:  README.md
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  test_tree
  untracked:  test_objects.c
  untracked:  Makefile
  untracked:  gen_tree_driver
  untracked:  tree.h
  untracked:  index.h
```

3b

```
adiga@adiga:~/Desktop/OS_Orange/os-u4-orange-problem$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776687666 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776687666 6 file2.txt
```

Phase 4

4a

```
adiga@adiga:~/Desktop/OS_Orange/os-u4-orange-problem$ ./pes log
commit 5a9db774352e7f5c8b2bb333451defc22bff059320800bbcd690015ec07fd862
Author: PES1UG24CS015 <PES1UG24CS015>
Date:   1776688063

    Add farewell

commit b7bbf00b8c8b784bf8c794693dc2fe97a1c35050910f99d4d3071e9e31de416c
Author: PES1UG24CS015 <PES1UG24CS015>
Date:   1776688063

    Add world

commit bb25083a445f1e6029319e88895097742f6d4023afb47bde006d26569555229d
Author: PES1UG24CS015 <PES1UG24CS015>
Date:   1776688063

    Initial commit
```

4b

```
adiga@adiga:~/Desktop/OS_Orange/os-u4-orange-problem$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/5a/9db774352e7f5c8b2bb333451defc22bff059320800bbcd690015ec07fd862
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/b7/bbf00b8c8b784bf8c794693dc2fe97a1c35050910f99d4d3071e9e31de416c
.pes/objects/bb/25083a445f1e6029319e88895097742f6d4023afb47bde006d26569555229d
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4c

```
adiga@adiga:~/Desktop/OS_Orange/os-u4-orange-problem$ cat .pes/refs/heads/main
cat .pes/HEAD
5a9db774352e7f5c8b2bb333451defc22bff059320800bbcd690015ec07fd862
ref: refs/heads/main
```

Phase 5 & 6

Q5.1

- Read target commit from .pes/refs/heads/<branch>.
- Validate no unsafe local changes (conflict check first).
- Update working directory to target tree:
- write/update tracked files from blob objects
- create/remove directories as needed
- remove tracked paths not in target tree
- apply file modes
- Rewrite .pes/index to match checked-out snapshot.
- Update .pes/HEAD to ref: refs/heads/<branch> atomically.
- Complexity: preventing data loss with uncommitted/untracked files, handling file↔directory path conflicts, and making multi-file updates crash-safe.

Q5.2

- For each tracked path in index:
- compare working file vs index metadata (mtime/size/existence)
- if changed, hash working file and compare with index hash
- Build path maps for current commit tree and target commit tree from object store.
- Refuse checkout if a locally modified tracked path would be changed/removed by target branch

Q5.3

- HEAD points directly to a commit, not a branch name.
- New commits are created normally but no branch moves forward.
- If user checks out another branch, those commits may become unreachable.
- Recovery: create a branch at that commit (or recover via reflog, then branch).

Q6.1

Use mark-and-sweep:

Mark:

- roots = all branch refs (and HEAD)
- traverse commit graph (parent + tree), then tree graph (subtrees + blobs)
- store visited hashes in a hash set

Sweep:

- scan .pes/objects
- delete objects not in reachable set

Efficient structure: hash set of object IDs (O(1) lookup).

Scale estimate (100k commits, 50 branches): typically around 100k unique commits plus related trees/blobs, usually a few hundred thousand objects total visited depending on churn.

Q6.2

Danger:

- Commit writes new objects first.
- Before ref update, those objects are temporarily unreachable from refs.
- Concurrent GC marks from refs, treats new objects as garbage, deletes them.
- Commit updates ref to now-missing objects -> corruption.

How Git avoids it:

- reference locking/transactions
- conservative pruning (age/grace period)
- reflogs as additional roots
- atomic pack/ref update practices to avoid deleting in-flight objects.